

Lab Exercise

HDFS and MapReduce to Analytics and Visualization Using Airline Sentiment Data

This lab exercise uses the Twitter US Airline Sentiment dataset. It is a real dataset of airline-related tweets. Public descriptions note that it contains tweets about major U.S. airlines from February 2015, with sentiment labels and negative-reason annotations, and that a commonly used public CSV mirror is approximately 3.26 MB. The CSV includes fields such as `airline\_sentiment`, `negativereason`, `airline`, `text`, and `tweet\_created`, which makes it suitable for both MapReduce and follow-up analytics.

Data source: <https://www.kaggle.com/datasets/crowdflower/twitter-airline-sentiment>

Official Apache Hadoop documentation describes HDFS as distributed, fault-tolerant storage for large datasets, and MapReduce as a framework for processing very large datasets in parallel across a cluster.

<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

Lab Objectives:

This lab is designed to understand **MapReduce in the broader scope of Big Data Analytics**.

By the end of the lab, students should see the full pipeline:

real dataset → distributed storage → MapReduce summarization → analytics → visualization

This lab is an extension of the previous HDFS storage lab exercise, which used small or dummy data to simulate HDFS storage.

Case study

Business scenario

Airlines receive large amounts of customer feedback through X (Twitter) and other online channels. Management wants to understand:

- Are customers mostly positive, neutral, or negative?
- Which complaint reason appears most often?
- Which airline receives more negative feedback?
- What issue should management prioritize?

Why these matters

If the data volume becomes very large, it is difficult to process manually or on one machine. Hadoop provides a distributed way to store and process such data. The airline sentiment dataset is widely used for sentiment analysis practice and includes the metadata needed for this lab.

Part 1. Start Docker and open the CLI

Step 1. Turn on Docker Desktop

Open **Docker Desktop** on your computer and wait until it shows that Docker is running.

Step 2. Open your terminal

On Windows, open: Command Prompt

Step 3. Go to your Hadoop project folder

Move into the folder where your `docker-compose.yml` file is located.

Example:

```
cd C:\ict329\ecosystem
```

If your folder is different, use your own path.

Step 4. Start the Hadoop services

Run this in your **host terminal**:

```
docker compose up -d
```

Explanation

This command starts the Hadoop ecosystem in the background.

What you should observe

Docker will start the containers for services such as:

- NameNode
- DataNode
- ResourceManager
- NodeManager

Step 5. Check that the containers are running

Run: `docker ps`

What you should observe

Running containers related to Hadoop, such as `namenode`.

If the containers are not running, you should wait a little longer and try again.

Step 6. Open the Hadoop command-line interface

Run this in the **host terminal**:

```
docker exec -it namenode bash
```

Explanation

This command opens a shell inside the Hadoop NameNode container.

What you should observe

The prompt will change to something like:

```
root@xxxxxxxx:/#
```

Now you are working **inside the container**.

Step 7. Check that HDFS is available

Inside the container, run: `hdfs dfs -ls /`

Explanation

This lists the top-level folders in HDFS.

If you want to delete a particular folder (i.e., week05), use this command:

```
hdfs dfs -rm -r /week05
```

Why these matters

HDFS is Hadoop's distributed file system. It stores data across the cluster rather than only on one machine.

Part 2. Retrieve the real dataset

Step 8. Create a working folder on the container

Inside the container, run:

```
mkdir -p /tmp/airline_lab  
cd /tmp/airline_lab
```

Explanation

This creates a temporary working folder for the lab files.

Step 9. Download the real dataset

Run (type/copy-paste in one line):

```
curl -L -o Tweets.csv  
https://raw.githubusercontent.com/ruchitgandhi/Twitter-Airline-Sentiment-Analysis/refs/heads/master/Tweets.csv
```

check if the file is downloaded complete by using

```
ls or ls -l
```

Explanation

This downloads the public airline sentiment dataset into the current folder.

What you should observe

A file named `Tweets.csv` will be created.

Step 10. Check the first line of the dataset

Run: `head -n 1 Tweets.csv`

What you should observe

You should see column names similar to:

```
tweet_id,airline_sentiment,...,negativereason,...,airline,...,text,...,tweet_created,...
```

Explanation

This confirms that the dataset contains fields for:

- sentiment
- complaint reason
- airline
- tweet text
- tweet date/time

These fields make the dataset useful for both MapReduce and later analytics.

Step 11. Inspect the first few rows

Run: `head -n 5 Tweets.csv`

Part 3. Put the dataset into HDFS

Step 12. Create a folder in HDFS

Run: `hdfs dfs -mkdir /airline_lab`

Explanation

This creates a distributed folder in HDFS for the lab.

Step 13. Upload the dataset to HDFS

Run:

```
hdfs dfs -put /tmp/airline_lab/Tweets.csv /airline_lab
```

Explanation

This copies the dataset from the container's local file system into HDFS.

Step 14. Verify the upload

Run: `hdfs dfs -ls /airline_lab`

What you should observe

There should a file name `Tweets.csv` inside `/airline_lab`.

Step 15. View a few lines from HDFS

Run:

```
hdfs dfs -cat /airline_lab/Tweets.csv | head -n 3
```

Explanation

This proves that the file is now stored in HDFS and can be read by Hadoop.

Concept learned

At this point, you have completed the **storage stage** of the Big Data pipeline.

Part 4. Prepare simplified files for MapReduce

Why preparation is needed

It is better **not** to run MapReduce directly on the raw tweet text first.

Reason

CSV tweet text can contain:

- commas
- quotation marks
- irregular formatting

That makes the lesson harder than necessary. Instead, we will extract:

- one sentiment label per line
- one complaint reason per line
- a smaller analytics subset for later analysis

Step 16. Run Python to prepare simpler files

Inside the container, run:

```
python3 - <<'PY'
import pandas as pd

df = pd.read_csv('/tmp/airline_lab/Tweets.csv')

df['airline_sentiment'].dropna().to_csv(
    '/tmp/airline_lab/sentiment_labels.txt',
    index=False,
    header=False
)

df['negativereason'].dropna().to_csv(
    '/tmp/airline_lab/negative_reasons.txt',
    index=False,
    header=False
)

df[['airline', 'airline_sentiment', 'negativereason',
'tweet_created']].to_csv(
    '/tmp/airline_lab/airline_analytics_subset.csv',
    index=False
)

print("Files created successfully.")
PY
```

Explanation

This creates three files:

- sentiment_labels.txt for counting positive, neutral, and negative labels
- negative_reasons.txt for counting complaint reasons
- airline_analytics_subset.csv for analytics and visualization

Important note:

If you encounter the error `bash: python3: command not found`, it means that Python is not installed in your container. In this case, you can use a separate notebook environment such as Google Colab or Jupyter Notebook, or install Python3 inside your container by following the steps below.

Inside the container, run this first:

```
cat /etc/os-release
which apt
```

If apt is found, then run:

```
apt update
apt install -y python3 python3-pip
pip3 install pandas
python3 --version
python3 -c "import pandas; print(pandas.__version__)"
```

If the installation still fails, use Google Colab instead (the fastest option). If you successfully install Python in your Docker container, redo Step 16 and then continue to Step 17.

To avoid confusion, the environments are separated clearly as follows:

- **Part A = Google Colab**
 - **Part B = Docker container**
-

Part A Google Colab: Prepare the Files for MapReduce and Analytics

Goal of this part

In this part, we will use **Google Colab** to:

1. download the real airline sentiment dataset
2. inspect the dataset
3. create three simplified files:
 - o sentiment_labels.txt
 - o negative_reasons.txt
 - o airline_analytics_subset.csv
4. download those files to their own computer

These files will later be copied into the Hadoop Docker container.

Step A1. Open Google Colab

Open your browser and go to **Google Colab**.

Create a **new notebook**.

Step A2. Create a working folder in Colab

In a new code cell, run:

```
import os

os.makedirs('/content/airline_lab', exist_ok=True)
print("Folder created:", '/content/airline_lab')
```

Explanation

This creates a folder inside Google Colab where the dataset and output files will be stored.

Step A3. Download the real dataset

In a new code cell, run:

```
import urllib.request

url = "https://raw.githubusercontent.com/ruchitgandhi/Twitter-Airline-Sentiment-Analysis/refs/heads/master/Tweets.csv"
output_path = "/content/airline_lab/Tweets.csv"

urllib.request.urlretrieve(url, output_path)
print("Downloaded to:", output_path)
```

Explanation

This downloads the real airline sentiment dataset into the Colab working folder.

Step A4. Check that the dataset exists

Run:

```
import os

print(os.path.exists('/content/airline_lab/Tweets.csv'))
print(os.path.getsize('/content/airline_lab/Tweets.csv'))
```

Explanation

This checks whether the file exists and shows its size in bytes.

Step A5. Inspect the first few rows of the dataset

Run:

```
import pandas as pd

df = pd.read_csv('/content/airline_lab/Tweets.csv')
df.head()
```

Explanation

This loads the dataset into Pandas and displays the first five rows.

Observe that the dataset contains real fields such as:

- airline_sentiment
- negativereason
- airline
- text
- tweet_created

Step A6. Check the column names

Run:

```
print(df.columns.tolist())
```

Explanation

This helps students clearly see the available variables in the dataset.

Step A7. Create the simplified files

Run:

```
df['airline_sentiment'].dropna().to_csv(
    '/content/airline_lab/sentiment_labels.txt',
    index=False,
    header=False
)

df['negativereason'].dropna().to_csv(
    '/content/airline_lab/negative_reasons.txt',
    index=False,
    header=False
)

df[['airline', 'airline_sentiment', 'negativereason',
    'tweet_created']].to_csv(
    '/content/airline_lab/airline_analytics_subset.csv',
    index=False
)

print("Files created successfully.")
```

Explanation

This creates three files for the next stages of the lab:

- sentiment_labels.txt
used for MapReduce sentiment counting
- negative_reasons.txt
used for MapReduce complaint-reason counting
- airline_analytics_subset.csv
used later for analytics and visualization

Step A8. Check that the files were created in Colab

Run:

```
import os

files = os.listdir('/content/airline_lab')
print(files)
```

Explanation

Students should see:

- Tweets.csv
- sentiment_labels.txt
- negative_reasons.txt
- airline_analytics_subset.csv

Step A9. Preview the sentiment labels file

Run:

```
with open('/content/airline_lab/sentiment_labels.txt', 'r', encoding='utf-8') as f:
    for _ in range(10):
        print(f.readline().strip())
```

Explanation

You should see lines such as:

```
neutral
positive
neutral
negative
```

This confirms that the file contains one clean label per line.

Step A10. Preview the negative reasons file

Run:

```
with open('/content/airline_lab/negative_reasons.txt', 'r', encoding='utf-8') as f:
    for _ in range(10):
        print(f.readline().strip())
```

Explanation

You should see complaint reasons such as delayed flight, customer service issue, or baggage-related issues.

Step A11. Preview the analytics subset file

Run:

```
subset_df =
pd.read_csv('/content/airline_lab/airline_analytics_subset.csv')
subset_df.head()
```

Explanation

This confirms that the smaller analytics dataset was created correctly.

Step A12. Download the files from Colab to your computer

Run:

```
from google.colab import files

files.download('/content/airline_lab/Tweets.csv')
files.download('/content/airline_lab/sentiment_labels.txt')
files.download('/content/airline_lab/negative_reasons.txt')
files.download('/content/airline_lab/airline_analytics_subset.csv')
```

Explanation

This downloads the files from Colab to your own computer.

You should save them in a known local folder, for example:

```
C:\airline_lab\
```

Files to save:

- Tweets.csv
 - sentiment_labels.txt
 - negative_reasons.txt
 - airline_analytics_subset.csv
-

Part B Copy the files into the Docker container

After downloading the files from Colab, we must move them into the Hadoop container.

This part is done in **Windows CMD / PowerShell**, not in Colab.

Step B13. Open Windows CMD or PowerShell

Open:

- Command Prompt
- PowerShell
- Windows Terminal

Go to the folder where your downloaded files were saved.

Example:

```
cd C:\airline_lab
```

Note: If you are still in `root@a0320d6c7ca6:/tmp/airline_lab#`, you can exit by pressing `Ctrl + Z`, and then go to the folder where your downloaded files are saved.

Example: `cd C:\airline_lab`

Step B14. Create the folder inside the container

Run:

```
docker exec -it namenode bash -c "mkdir -p /tmp/airline_lab"
```

Explanation

This creates the `/tmp/airline_lab` folder inside the `namenode` container.

Step B15. Copy the files into the container

Run the following command (assuming the original file is located in `C:\airline_lab`):

Note: run the following command as one line for each

- `docker cp C:\airline_lab\Tweets.csv namenode:/tmp/airline_lab/Tweets.csv`
- `docker cp C:\airline_lab\sentiment_labels.txt namenode:/tmp/airline_lab/sentiment_labels.txt`
- `docker cp C:\airline_lab\negative_reasons.txt namenode:/tmp/airline_lab/negative_reasons.txt`
- `docker cp C:\airline_lab\airline_analytics_subset.csv namenode:/tmp/airline_lab/airline_analytics_subset.csv`

Explanation

This copies the downloaded files from your Windows computer into the Docker container.

Step B16. Enter the container

Run: `docker exec -it namenode bash`

Then move to the working folder: `cd /tmp/airline_lab`

At this point, your prompt should look similar to:

```
root@xxxxxxx:/tmp/airline_lab#
```

Step 17. Check that the files were created

Run: `ls -l /tmp/airline_lab`

What you should observe

You should see:

- `Tweets.csv`
 - `sentiment_labels.txt`
 - `negative_reasons.txt`
 - `airline_analytics_subset.csv`
-

Part 5. Upload the prepared files to HDFS

Step 18. Upload the simplified files

Run:

```
hdfs dfs -put /tmp/airline_lab/sentiment_labels.txt /airline_lab
hdfs dfs -put /tmp/airline_lab/negative_reasons.txt /airline_lab
hdfs dfs -put /tmp/airline_lab/airline_analytics_subset.csv /airline_lab
```

Step 19. Verify the upload

Run: `hdfs dfs -ls /airline_lab`

What you should observe

All four files should now appear in HDFS.

Part 6. Run MapReduce for sentiment distribution

Step 20. Find the MapReduce examples JAR

Run: `find / -name "*mapreduce*examples*.jar" 2>/dev/null`

Explanation

Different Docker Hadoop images store the MapReduce example JAR in different paths. This command finds the correct one.

Example result

You may see something like:

```
/usr/local/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
```

You must copy the path that actually get.

Step 21. Run WordCount on sentiment labels

Replace the path below with the one found on your system:

```
hadoop jar /usr/local/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount /airline_lab/sentiment_labels.txt /airline_lab/output_sentiment
```

Explanation

This runs a MapReduce WordCount job on the file that contains one sentiment label per line.

What MapReduce is doing

Observe:

- the input file is split
- mappers process parts of the file
- identical keys are grouped together
- reducers sum the counts

This is the core MapReduce lesson. Apache's MapReduce tutorial describes the same overall model: map, shuffle/sort, then reduce.

Step 22. View the sentiment result

Run: `hdfs dfs -cat /airline_lab/output_sentiment/part-r-00000`

What you should observe

Observe counts for:

- negative
- neutral
- positive

Explanation

This is the first meaningful MapReduce output in the lab.

Part 7. Run MapReduce for complaint reasons

Step 23. Run WordCount on complaint reasons

Again, replace the JAR path if needed:

```
hadoop jar /usr/local/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount /airline_lab/negative_reasons.txt /airline_lab/output_reasons
```

Explanation

This counts how many times each negative complaint reason appears in the dataset.

Step 24. View the complaint reason result

Run:

```
hdfs dfs -cat /airline_lab/output_reasons/part-r-00000 | head -n 20
```

Explanation

This is more business-relevant than generic word counting because the output now summarizes actual complaint categories from real airline feedback data.

Part 8. Transition from MapReduce to analytics

Why this step matters

MapReduce produces **aggregated counts**.

Analytics goes further and asks:

- Which sentiment dominates overall?
- Which airline receives more negative tweets?
- Which complaint reason is most common?
- Do some airlines show different complaint patterns?

This is the point where the lesson expands from Hadoop infrastructure into Big Data Analytics.

Part 9. Analytics Extension Using Google Colab and Pandas

Introduction

In this lab, I assume that **most PCs in this lab do not have Python installed inside the Hadoop container**. Because of that, the following exercise is designed using **Google Colab**.

If your container already has **Python and Pandas** installed, you may run the same analytics steps using the **CLI** or **Jupyter inside the container**. However, to make the exercise easier and more consistent for everyone, we use **Google Colab** for the following part.

So, in this section:

- **HDFS and MapReduce** are still done in the Docker Hadoop environment
- **analytics and visualization** are done in **Google Colab**

This makes the lab easier for beginner students while still preserving the full Big Data Analytics workflow.

Goal of this part

In this section, we will use **Google Colab** to:

1. load the prepared analytics dataset
2. perform simple data analytics using Pandas
3. answer business questions from the airline dataset
4. create visualizations for interpretation

The file used in this section is:

```
airline_analytics_subset.csv
```

This file was created earlier from the real airline sentiment dataset.

Step 25. Open Google Colab

Open your browser and go to **Google Colab**.

Create a **new notebook**.

Step 26. Upload the analytics subset file into Colab

In a new Colab code cell, run:

```
from google.colab import files
uploaded = files.upload()
```

What you should do

Upload this file from your computer:

```
airline_analytics_subset.csv
```

Explanation

This makes the prepared dataset available in the Colab environment.

Step 27. Load the subset file

In a new code cell, run:

```
import pandas as pd
df = pd.read_csv('airline_analytics_subset.csv')
df.head()
```

Explanation

This loads the smaller analytics dataset into Pandas and displays the first few rows.

Observe columns such as:

- airline
- airline_sentiment
- negativereason

- `tweet_created`

Step 28. Find the overall sentiment distribution

Run:

```
sentiment_counts = df['airline_sentiment'].value_counts()
print(sentiment_counts)
```

Explanation

This shows how many tweets are:

- negative
- neutral
- positive

overall in the dataset.

This helps to understand the general sentiment pattern.

Step 29. Find sentiment by airline

Run:

```
sentiment_by_airline = pd.crosstab(df['airline'], df['airline_sentiment'])
print(sentiment_by_airline)
```

Explanation

This compares airlines by sentiment category.

Step 30. Find the most common negative reasons

Run:

```
top_reasons = df['negativereason'].dropna().value_counts().head(10)
print(top_reasons)
```

Step 31. Find negative reasons by airline

Run:

```
reasons_by_airline = pd.crosstab(df['airline'], df['negativereason'])
print(reasons_by_airline)
```

Step 32. Create a daily tweet count

Run:

```
df['tweet_created'] = pd.to_datetime(df['tweet_created'], errors='coerce')
df['date'] = df['tweet_created'].dt.date
daily_counts = df.groupby('date').size()
print(daily_counts.head())
```

Part 10. Visualization Extension in Google Colab

Why visualization is important

Visualization turns tables and counts into insights that managers can understand quickly.

Instead of reading a long table of numbers, decision-makers can interpret charts more easily.

Step 33. Import Matplotlib

In a new code cell, run:

```
import matplotlib.pyplot as plt
```

Step 34. Create a bar chart for sentiment distribution

Run:

```
sentiment_counts.plot(kind='bar')
plt.title('Overall Sentiment Distribution')
plt.xlabel('Sentiment')
plt.ylabel('Count')
plt.show()
```

What this shows

A bar chart showing the number of:

- negative tweets
- neutral tweets
- positive tweets

Step 35. Create a horizontal bar chart for top negative reasons

Run:

```
top_reasons.plot(kind='barh')
plt.title('Top Negative Reasons')
plt.xlabel('Count')
plt.ylabel('Negative Reason')
plt.show()
```

What this shows

This chart shows the most common complaint categories in the dataset.

Step 36. Create a line chart for daily tweet volume

Run:

```
plt.figure(figsize=(12,5))
daily_counts.plot(kind='line')
plt.title('Daily Tweet Volume')
plt.xlabel('Date')
plt.ylabel('Number of Tweets')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

What this shows

This line chart shows how tweet volume changes over time.

Optional extra chart using stacked bar chart

Run:

```
sentiment_by_airline.plot(kind='bar', stacked=True)
plt.title('Sentiment by Airline')
plt.xlabel('Airline')
plt.ylabel('Count')
plt.show()
```

What this shows

This chart compares airlines across multiple sentiment categories in one figure.

Exercise Submission

All students should submit:

- All Python files from Google Colab
- A short interpretation that answers:
 - the main sentiment trend
 - the most common complaint
 - one operational recommendation for airlines

Please put **all** the **python files** and a short **interpretation file** in one **zip** file and submit to the google classroom.